ADVANCED DIGITAL LOGIC DESIGN

AAT

SUBMITTED BY - S SOUPARNIKA
USN - 1BF24EC126
SEMESTER AND SECTION - 5-B

Under the guidance of : **PROF. AJAYKUMAR DEVARAPALLI**

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

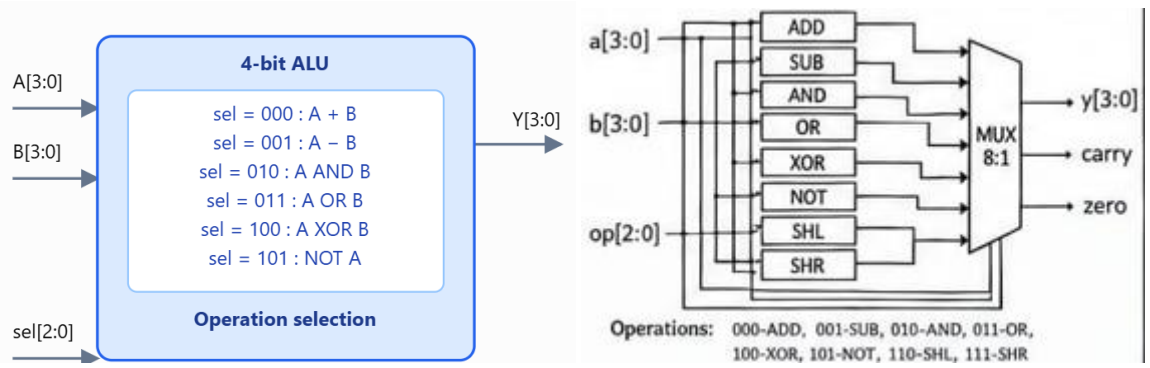
2026 – 2027

TABLE OF CONTENTS

SL NO	CONTENTS	PAGE NO.
1	ALU- ARITHMETIC LOGIC UNIT	3
2	COMPARATOR	5
3	4 BIT PARITY GENERATOR AND CHECKER	7
4	4 BIT BARREL SHIFTER	9
5	TIMER MODULE	11
6	RING COUNTER	13
7	JOHNSON COUNTER	15
8	SYNCHRONOUS COUNTER	17
9	ASYNCHRONOUS COUNTER	19
10	SHIFT REGISTER	21
11	DIVIDE BY N COUNTER	23
12	MEALY FSM	26
13	MOORE FSM	29
14	RANDOM SEQUENCE GENERATOR	32
15	CONCLUSION	34

1. ALU – ARITHMETIC LOGIC UNIT

DESIGN: The 4-bit Arithmetic Logic Unit is a combinational circuit designed using Verilog HDL. It takes two 4-bit inputs, A and B, and a 3-bit selection input, sel. The output is a 4-bit result Y.



000	Addition	Y = A + B
001	Subtraction	Y = A – B
010	Bitwise AND	Y = A & B
011	Bitwise OR	Y = A | B
100	Bitwise XOR	Y = A ^ B
101	Bitwise NOT	Y = ~A
110	Default	Y = 0000
111	Default	Y = 0000

VERILOG HDL DESIGN CODE:

```
module alu(
   input [3:0] A, B,
   input [2:0] sel,
   output reg [3:0] Y
);
always @(*) begin
   case(sel)
      3'b000: Y = A + B;
      3'b001: Y = A - B;
      3'b010: Y = A & B;
      3'b011: Y = A | B;
      3'b100: Y = A ^ B;
      3'b101: Y = ~A;
      default: Y = 4'b0000;
   endcase
end
endmodule
```

TESTBENCH CODE:

```
module tb;
  reg [3:0] A, B;
  reg [2:0] sel;
  wire [3:0] Y;
  alu dut(A, B, sel, Y);
  initial begin
    A = 5;
    B = 3;
    sel = 0; #10;
    sel = 1; #10;
    sel = 2; #10;
    sel = 3; #10;
    sel = 4; #10;
    sel = 5; #10;
    $finish;
  end
  initial
    $monitor("A=%d B=%d sel=%d Y=%d",
          A, B, sel, Y);
endmodule
```

SIMULATION RESULT:

```
module tb;
reg [3:0] A,B;
reg [2:0] sel;
wire [3:0] Y;
alu dut(A,B,sel,Y);
initial begin
A=4'd5; B=4'd3;
sel=0; #10;
sel=1; #10;
sel=2; #10;
sel=3; #10;
sel=4; #10;
sel=5; #10;
$finish;
end
```

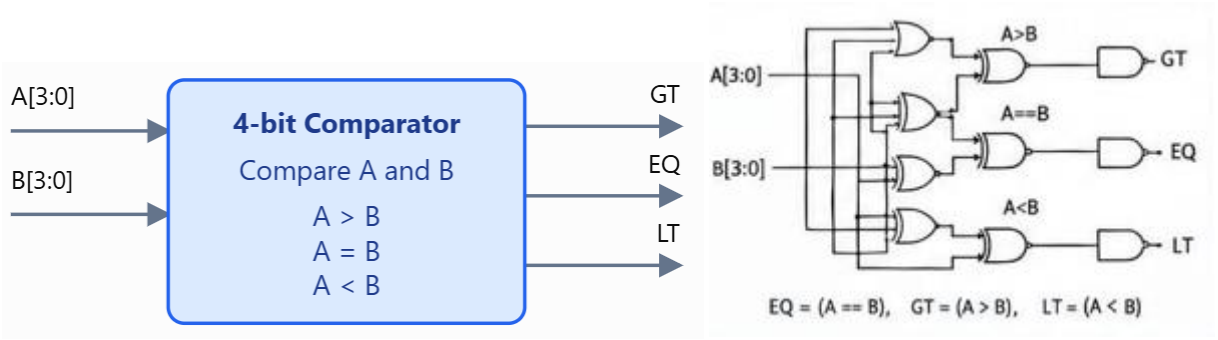
```
// Code your design here
module alu(
input [3:0] A,B,
input [2:0] sel,
output reg [3:0] Y
);
always @(*) begin
case(sel)
0: Y=A+B;
1: Y=A-B;
2: Y=A&B;
3: Y=A|B;
4: Y=A^B;
5: Y=~A;
default: Y=0;
```

Log | Share

```
[2026-10-25 18:31:27 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
A= 5 B= 3 sel=0 Y= 8
A= 5 B= 3 sel=1 Y= 2
A= 5 B= 3 sel=2 Y= 1
A= 5 B= 3 sel=3 Y= 7
A= 5 B= 3 sel=4 Y= 6
A= 5 B= 3 sel=5 Y=10
testbench.sv:16: $finish called at 60 (1s)
Done
```

2. COMPARATOR

DESIGN: This comparator compares two 4-bit inputs, A and B, and produces three outputs: GT greater than, EQ equal, and LT less than.



Example:

A	B	GT	EQ	LT
5	3	1	0	0
3	3	0	1	0
2	7	0	0	1

VERILOG HDL DESIGN CODE:

```
module comparator(
   input [3:0] A, B,
   output gt, eq, lt
);
assign gt = (A > B);
assign eq = (A == B);
assign lt = (A < B);

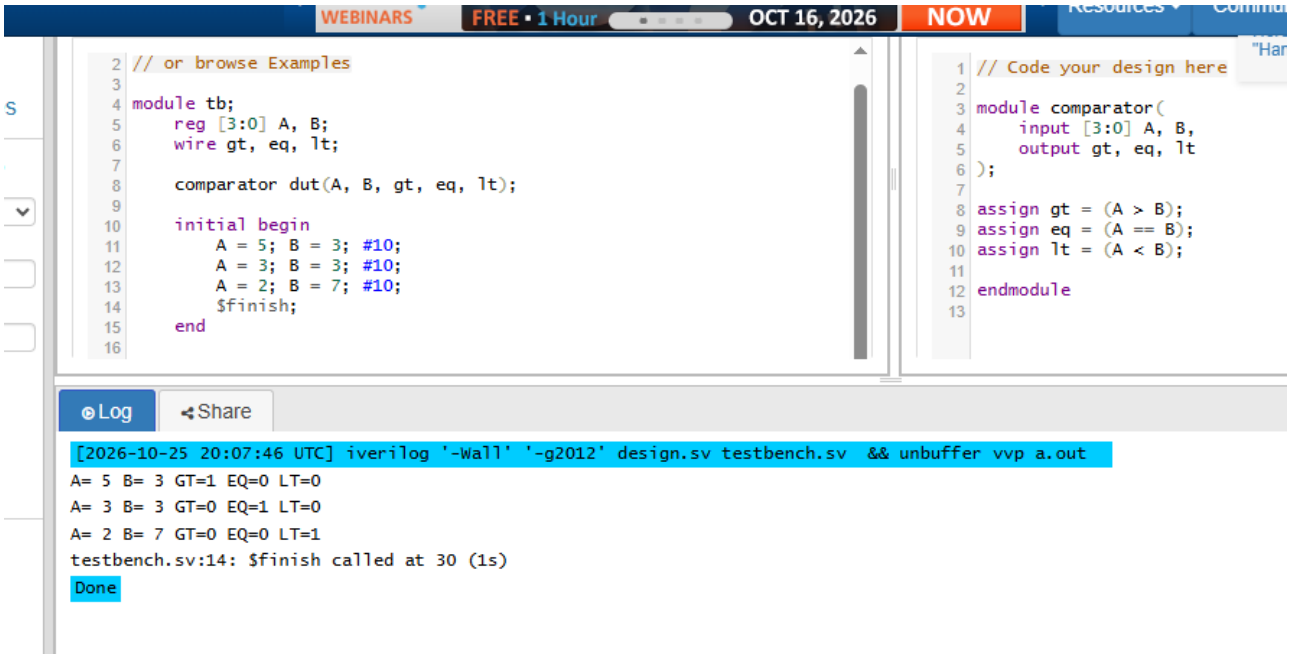
endmodule
```

TESTBENCH CODE:

```
module tb;
   reg [3:0] A, B;
   wire gt, eq, lt;
   comparator dut(A, B, gt, eq, lt);
   initial begin
      A = 5; B = 3; #10;
      A = 3; B = 3; #10;
      A = 2; B = 7; #10;
      $finish;
   end
   initial
      $monitor("A=%d B=%d GT=%b EQ=%b LT=%b",
            A, B, gt, eq, lt);
```

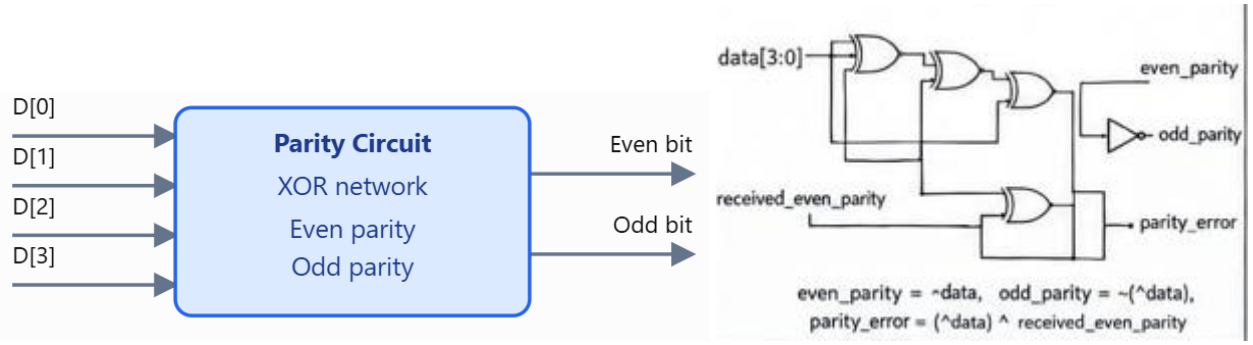
endmodule

SIMULATION RESULT:



3. 4-bit Parity Generator and Checker

DESIGN: The 4-bit parity generator generates even and odd parity bits from a 4-bit input using XOR operations. These parity bits help detect errors in digital data transmission.



Data	Received even parity	Generated even parity	Odd parity	Error
1011	1	1	0	0
1011	0	1	0	1
1111	0	0	1	0
1111	1	0	1	1

VERILOG HDL DESIGN CODE:

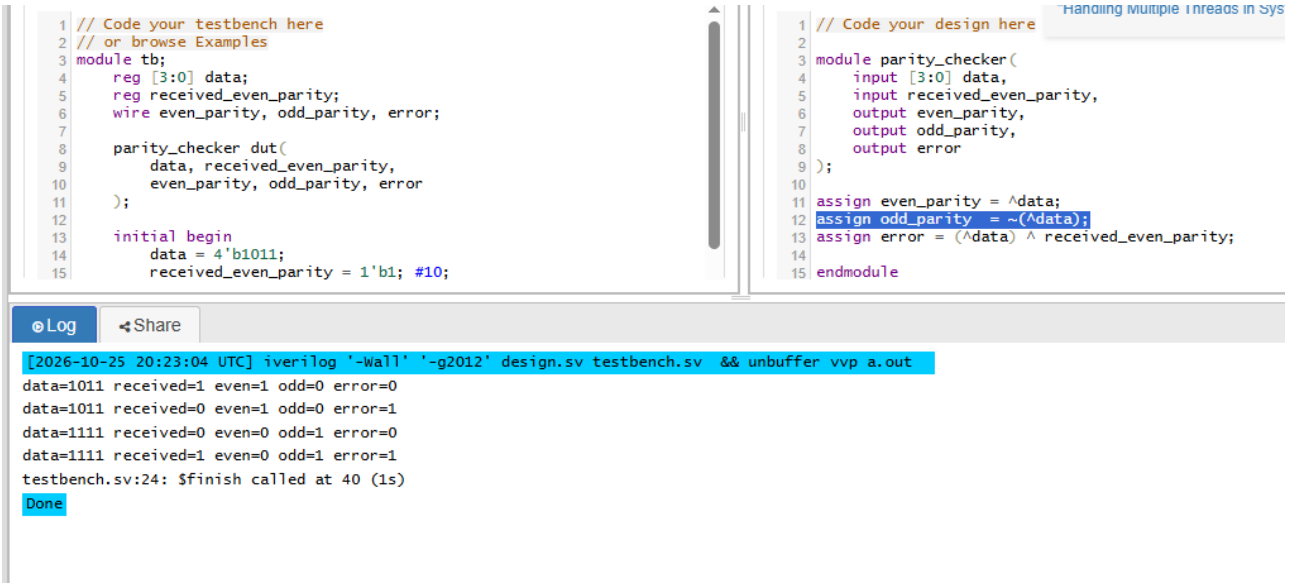
```
module parity_checker(
   input [3:0] data,
   input received_even_parity,
   output even_parity,
   output odd_parity,
   output error);
assign even_parity = ^data;
assign odd_parity  = ~(^data);
assign error = (^data) ^ received_even_parity;
endmodule
```

TESTBENCH CODE:

```
module tb;
   reg [3:0] data;
   reg received_even_parity;
   wire even_parity, odd_parity, error;
   parity_checker dut(
      data, received_even_parity,
      even_parity, odd_parity, error );
   initial begin
      data = 4'b1011;
      received_even_parity = 1'b1; #10;
      received_even_parity = 1'b0; #10;
      data = 4'b1111;
      received_even_parity = 1'b0; #10;
```

```
    received_even_parity = 1'b1; #10;
    $finish;
  end
  initial
    $monitor("data=%b received=%b even=%b odd=%b error=%b",
    data, received_even_parity, even_parity, odd_parity, error);
endmodule
```

SIMULATION RESULT:



```
// Code your testbench here
// or browse Examples
module tb;
    reg [3:0] data;
    reg received_even_parity;
    wire even_parity, odd_parity, error;

    parity_checker dut(
        data, received_even_parity,
        even_parity, odd_parity, error
    );

    initial begin
        data = 4'b1011;
        received_even_parity = 1'b1; #10;
```

```
// Code your design here

module parity_checker(
    input [3:0] data,
    input received_even_parity,
    output even_parity,
    output odd_parity,
    output error
);

assign even_parity = ^data;
assign odd_parity  = ~(^data);
assign error = (^data) ^ received_even_parity;

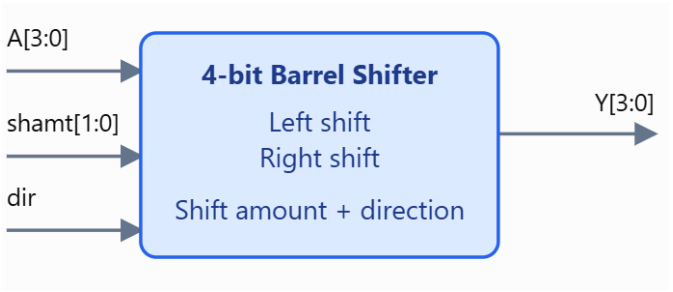
endmodule
```

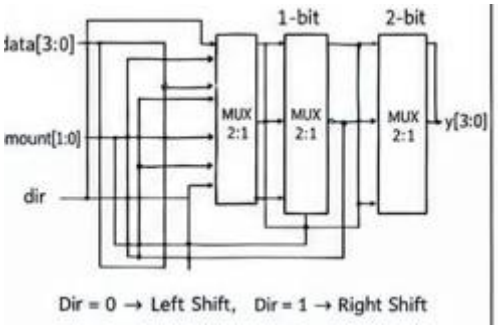
Log | Share

```
[2026-10-25 20:23:04 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
data=1011 received=1 even=1 odd=0 error=0
data=1011 received=0 even=1 odd=0 error=1
data=1111 received=0 even=0 odd=1 error=0
data=1111 received=1 even=0 odd=1 error=1
testbench.sv:24: $finish called at 40 (1s)
Done
```

4. 4-bit Barrel Shifter

DESIGN: The 4-bit barrel shifter shifts the input data left or right by a selected number of bit positions. The dir input selects the direction, while shamt specifies the shift amount.





Example:

A	dir	Shift amount	Y
1011	0	1	0110
1011	0	2	1100
1011	1	1	0101
1011	1	2	0010

Here, `dir = 0` means left shift and `dir = 1` means right shift.

VERILOG HDL DESIGN CODE:

```
module barrel(
   input [3:0] A,
   input [1:0] shamt,
   input dir,
   output [3:0] Y);
assign Y = dir ? (A >> shamt) : (A << shamt);
endmodule
```

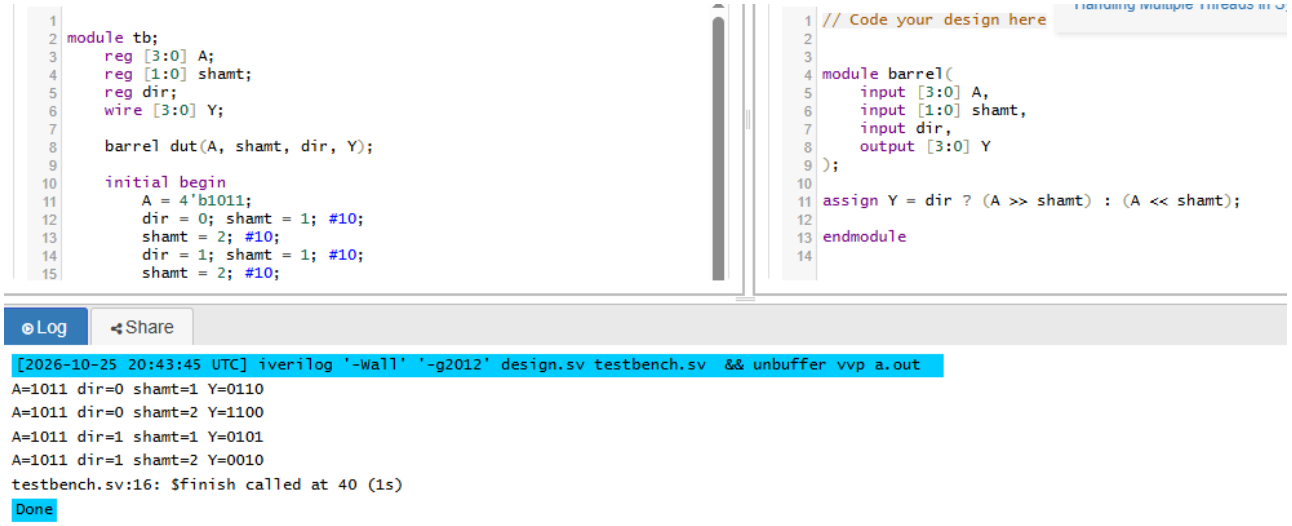
TESTBENCH CODE:

```
module tb;
   reg [3:0] A;
   reg [1:0] shamt;
   reg dir;
   wire [3:0] Y;
   barrel dut(A, shamt, dir, Y);
   initial begin
      A = 4'b1011;
      dir = 0; shamt = 1; #10;
      shamt = 2; #10;
      dir = 1; shamt = 1; #10;
      shamt = 2; #10;
```

```
    $finish;
  end


  initial
    $monitor("A=%b dir=%b shamt=%d Y=%b",
          A, dir, shamt, Y);
endmodule
```

SIMULATION RESULT:



```
module tb;
    reg [3:0] A;
    reg [1:0] shamt;
    reg dir;
    wire [3:0] Y;

    barrel dut(A, shamt, dir, Y);

    initial begin
        A = 4'b1011;
        dir = 0; shamt = 1; #10;
        shamt = 2; #10;
        dir = 1; shamt = 1; #10;
        shamt = 2; #10;
```

```
// Code your design here


module barrel(
    input [3:0] A,
    input [1:0] shamt,
    input dir,
    output [3:0] Y
);

assign Y = dir ? (A >> shamt) : (A << shamt);

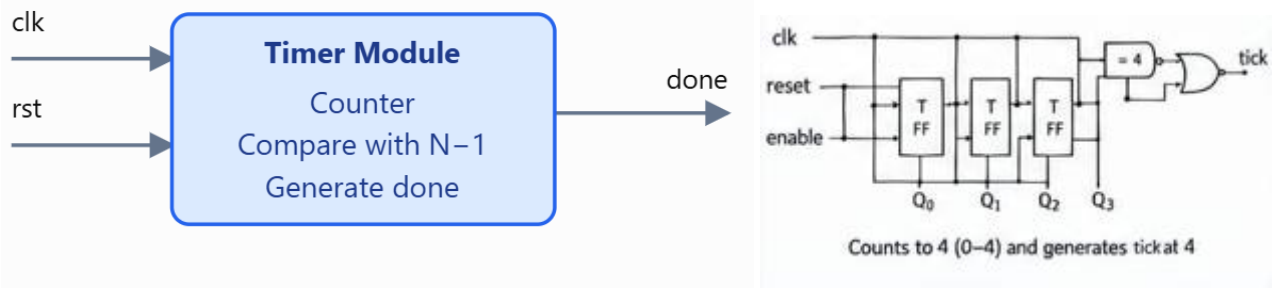
endmodule
```

Log Share

```
[2026-10-25 20:43:45 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
A=1011 dir=0 shamt=1 Y=0110
A=1011 dir=0 shamt=2 Y=1100
A=1011 dir=1 shamt=1 Y=0101
A=1011 dir=1 shamt=2 Y=0010
testbench.sv:16: $finish called at 40 (1s)
Done
```

5. Timer Module

DESIGN: The timer uses a clock-driven counter that increments on each rising clock edge. When the counter reaches N-1, it resets to zero and activates the done output for one clock cycle.



Example : N = 4. The done signal becomes 1 whenever the counter reaches N−1, then returns to 0 on the next clock edge.

Clock event	Reset (rst)	Counter (count)	done
Reset active	1	0	0
First rising edge after reset	0	1	0
Second rising edge	0	2	0
Third rising edge	0	3	0
Fourth rising edge	0	0	1
Fifth rising edge	0	1	0
Sixth rising edge	0	2	0
Seventh rising edge	0	3	0
Eighth rising edge	0	0	1

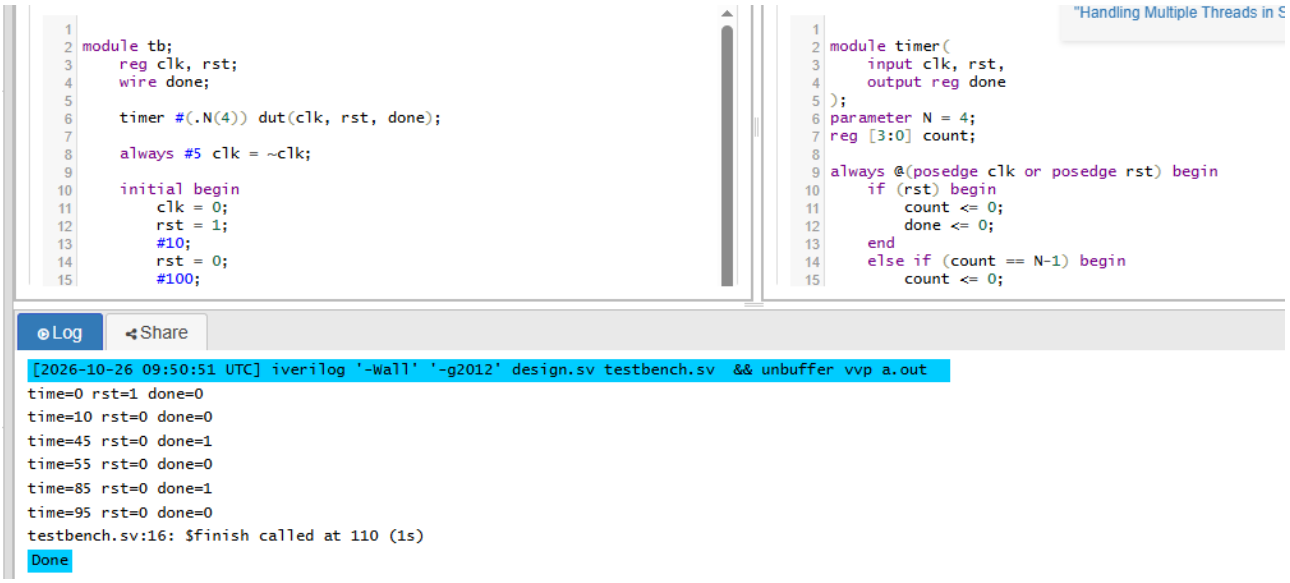
VERILOG HDL DESIGN CODE:

```
module timer(
   input clk, rst,
   output reg done);
parameter N = 4;
reg [3:0] count;
always @(posedge clk or posedge rst) begin
   if (rst) begin
      count <= 0;
      done <= 0;
   end
   else if (count == N-1) begin
      count <= 0;
      done <= 1;
   end
   else begin
      count <= count + 1;
      done <= 0;
   end
end
endmodule
```

TESTBENCH CODE:

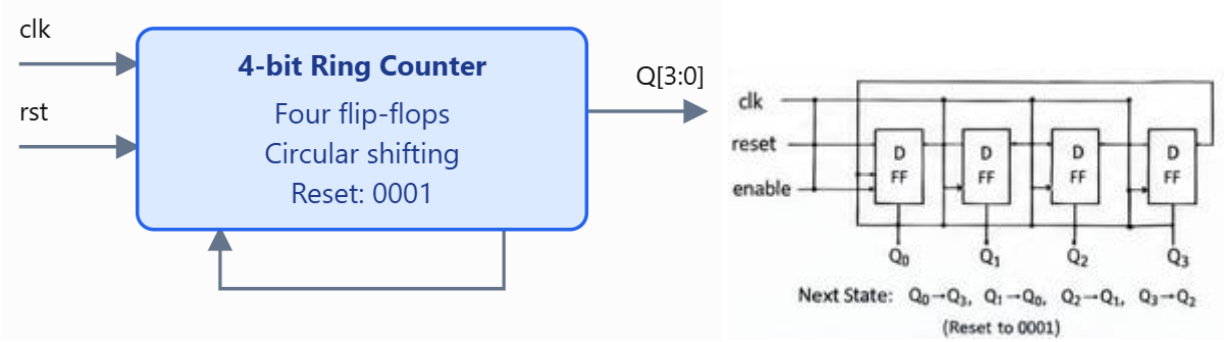
```
module tb;
   reg clk, rst;
   wire done;
   timer #(.N(4)) dut(clk, rst, done);
   always #5 clk = ~clk;
   initial begin
      clk = 0;
      rst = 1;
      #10;
      rst = 0;
      #100;
      $finish;
   end
   initial
      $monitor("time=%0t rst=%b done=%b",
            $time, rst, done);
endmodule
```

SIMULATION RESULT:



6. Ring Counter

DESIGN: A 4-bit ring counter consists of four flip-flops connected in a circular shift arrangement. On each rising clock edge, the stored bit pattern shifts by one position, with the last bit fed back to the first; reset initializes the counter to 0001.



Clock edge	Q[3:0]
Reset	0001
1	0010
2	0100
3	1000
4	0001
5	0010
6	0100

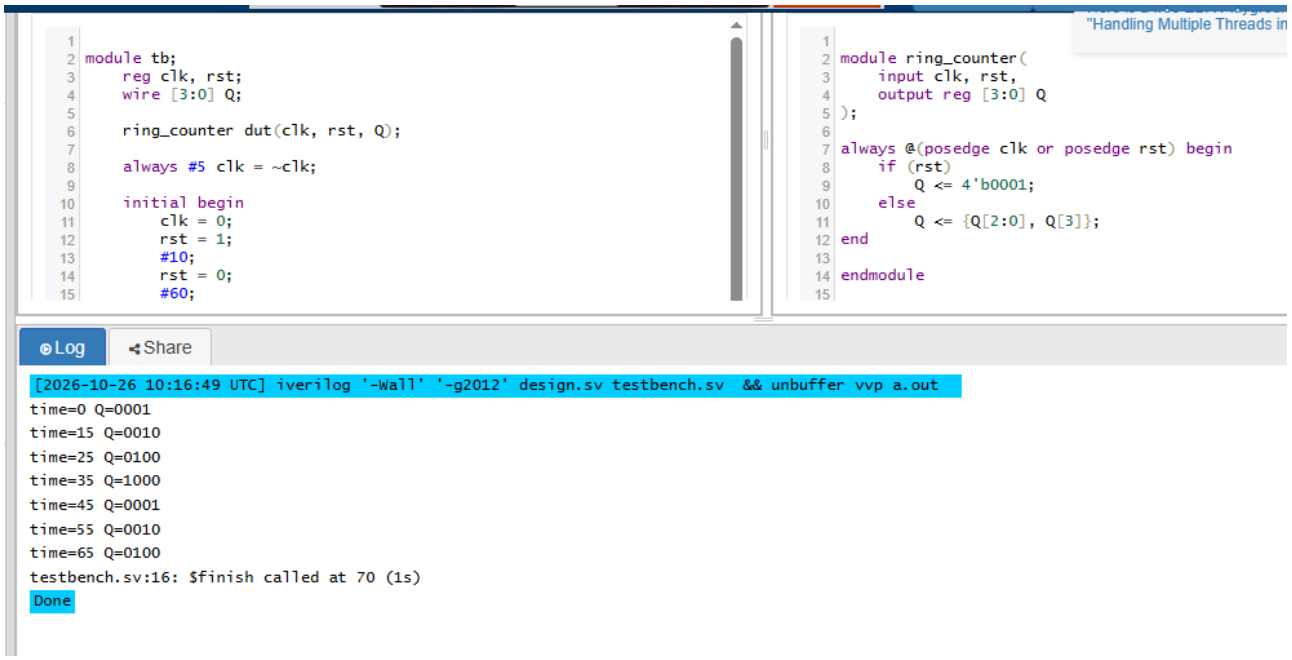
VERILOG HDL DESIGN CODE:

```
module ring_counter(
   input clk, rst,
   output reg [3:0] Q);
always @(posedge clk or posedge rst) begin
   if (rst)
      Q <= 4'b0001;
   else
      Q <= {Q[2:0], Q[3]};
end
endmodule
```

TESTBENCH CODE:

```
module tb;
   reg clk, rst;
   wire [3:0] Q;
   ring_counter dut(clk, rst, Q);
   always #5 clk = ~clk;
   initial begin
      clk = 0;
      rst = 1;
      #10;
      rst = 0;
      #60;
      $finish;
   end
   initial
      $monitor("time=%0t Q=%b", $time, Q);
endmodule
```

SIMULATION RESULT:



7. Johnson Counter

DESIGN: A 4-bit Johnson counter consists of four flip-flops connected as a shift register. The inverted output of the last flip-flop is fed back to the input, producing eight distinct states during normal operation.

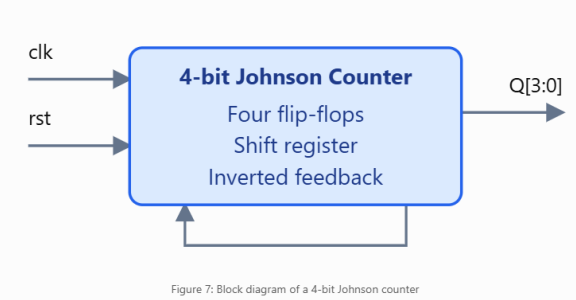
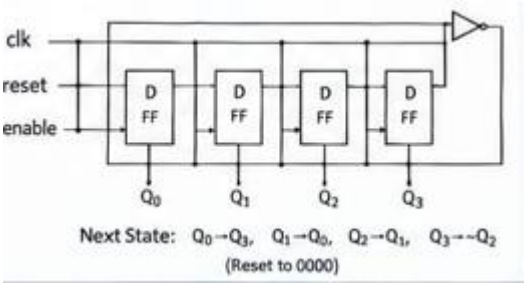


Figure 7: Block diagram of a 4-bit Johnson counter



Clock edge	Q[3:0]
Reset	0000
1	0001
2	0011
3	0111
4	1111
5	1110
6	1100
7	1000
8	0000

VERILOG HDL DESIGN CODE:

```
module johnson(
   input clk, rst,
   output reg [3:0] Q);
always @(posedge clk or posedge rst) begin
   if (rst)
      Q <= 4'b0000;
   else
      Q <= {Q[2:0], ~Q[3]};
end
endmodule
```

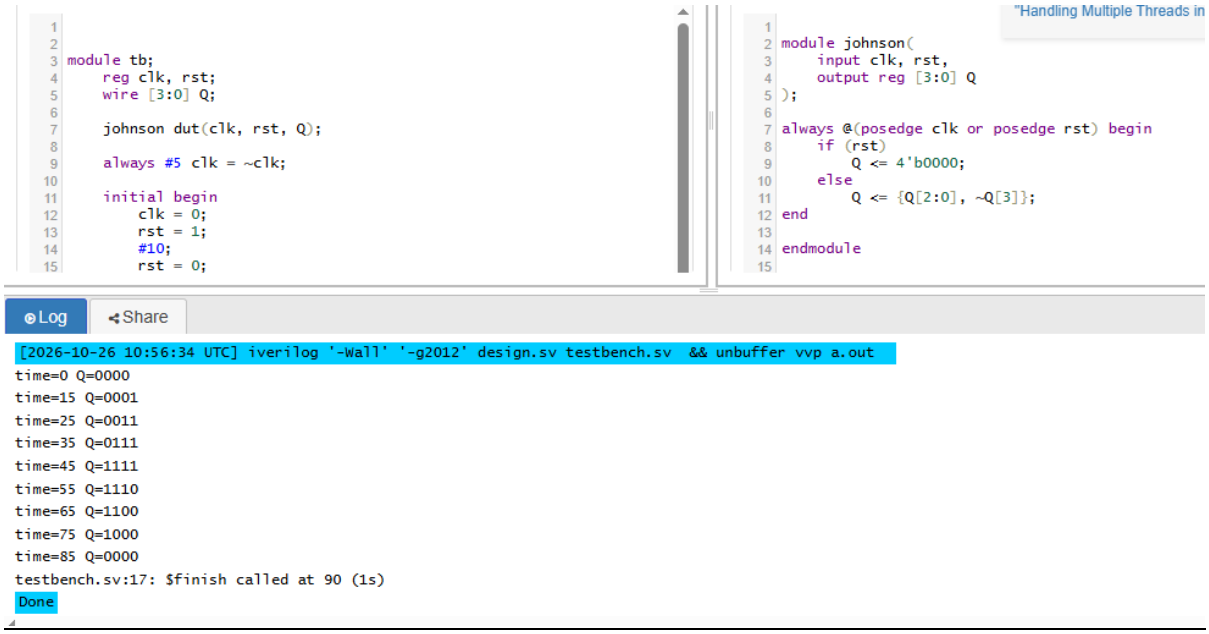
TESTBENCH CODE:

```
module tb;
```

```
    reg clk, rst;
    wire [3:0] Q;

    johnson dut(clk, rst, Q);
    always #5 clk = ~clk;
    initial begin
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #80;
        $finish;
    end
    initial
        $monitor("time=%0t Q=%b", $time, Q);
endmodule
```

SIMULATION RESULT:



```
module tb;
    reg clk, rst;
    wire [3:0] Q;

    johnson dut(clk, rst, Q);

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
```

```
module johnson(
    input clk, rst,
    output reg [3:0] Q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], ~Q[3]};
end

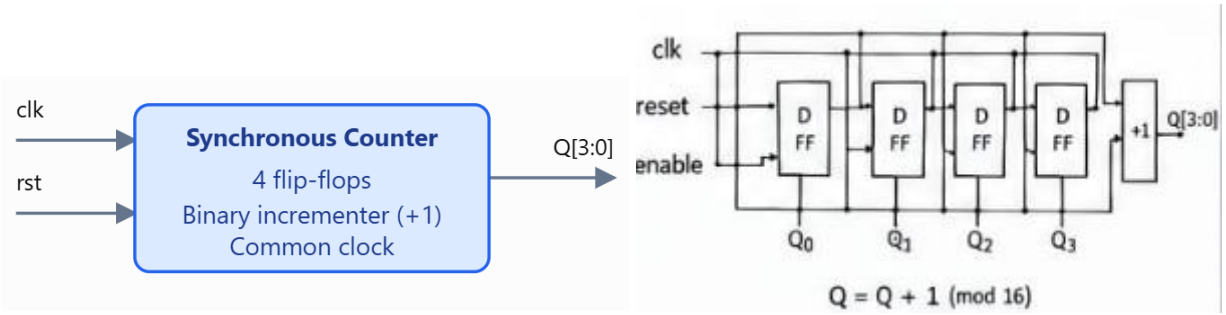
endmodule
```

Log | Share

```
[2026-10-26 10:56:34 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
time=0 Q=0000
time=15 Q=0001
time=25 Q=0011
time=35 Q=0111
time=45 Q=1111
time=55 Q=1110
time=65 Q=1100
time=75 Q=1000
time=85 Q=0000
testbench.sv:17: $finish called at 90 (1s)
Done
```

8. Synchronous Counter

DESIGN: A 4-bit synchronous counter uses four flip-flops driven by the same clock signal. On each rising clock edge, the count increases by one, and reset initializes the output to zero.



Clock edge	Output Q
Reset	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000

VERILOG HDL DESIGN CODE:

```
module sync_counter(
    input clk, rst,
    output reg [3:0] Q
);
always @(posedge clk or posedge rst) begin
    if (rst)
        Q <= 4'b0000;
    else
        Q <= Q + 1'b1;
end
endmodule
```

TESTBENCH CODE:

```
module tb;
    reg clk, rst;
    wire [3:0] Q;

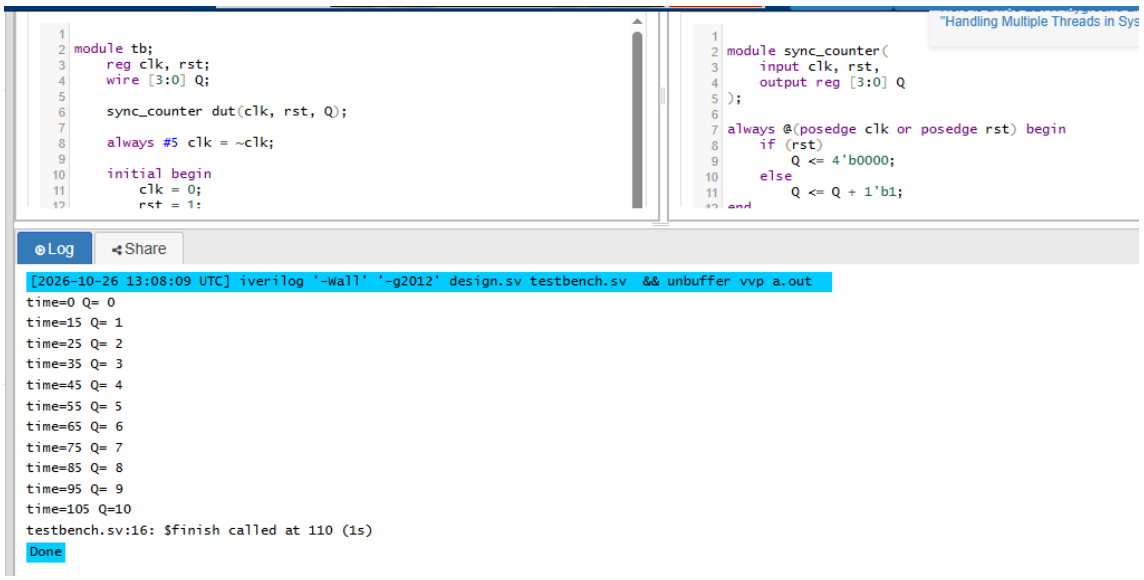
    sync_counter dut(clk, rst, Q);

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #100;
        $finish;
    end

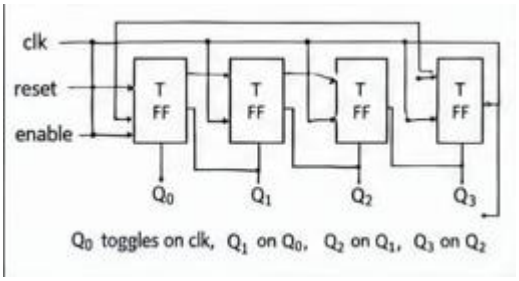
    initial
        $monitor("time=%0t Q=%d", $time, Q);
endmodule
```

SIMULATION RESULT:



9. Asynchronous Counter

DESIGN: A 4-bit asynchronous counter consists of four flip-flops in which the first flip-flop receives the external clock, while each subsequent flip-flop is triggered by the preceding stage's output. Therefore, the output bits do not change simultaneously.



Clock pulse	Q[3]	Q[2]	Q[1]	Q[0]	Output Q
Reset	0	0	0	0	0000
1	0	0	0	1	0001
2	0	0	1	0	0010
3	0	0	1	1	0011
4	0	1	0	0	0100
5	0	1	0	1	0101
6	0	1	1	0	0110
7	0	1	1	1	0111
8	1	0	0	0	1000
9	1	[illegible]	0	1	1001
10	1	0	1	0	1010

Verilog HDL design code:

```
module async_counter(
   input clk, rst,
   output reg [3:0] Q
);
always @(posedge clk or posedge rst)
   if (rst) Q[0] <= 0;
   else Q[0] <= ~Q[0];
always @(negedge Q[0] or posedge rst)
   if (rst) Q[1] <= 0;
   else Q[1] <= ~Q[1];
always @(negedge Q[1] or posedge rst)
   if (rst) Q[2] <= 0;
   else Q[2] <= ~Q[2];
always @(negedge Q[2] or posedge rst)
   if (rst) Q[3] <= 0;
   else Q[3] <= ~Q[3];
```

endmodule

TESTBENCH CODE:

```
module tb;
   reg clk, rst;
   wire [3:0] Q;

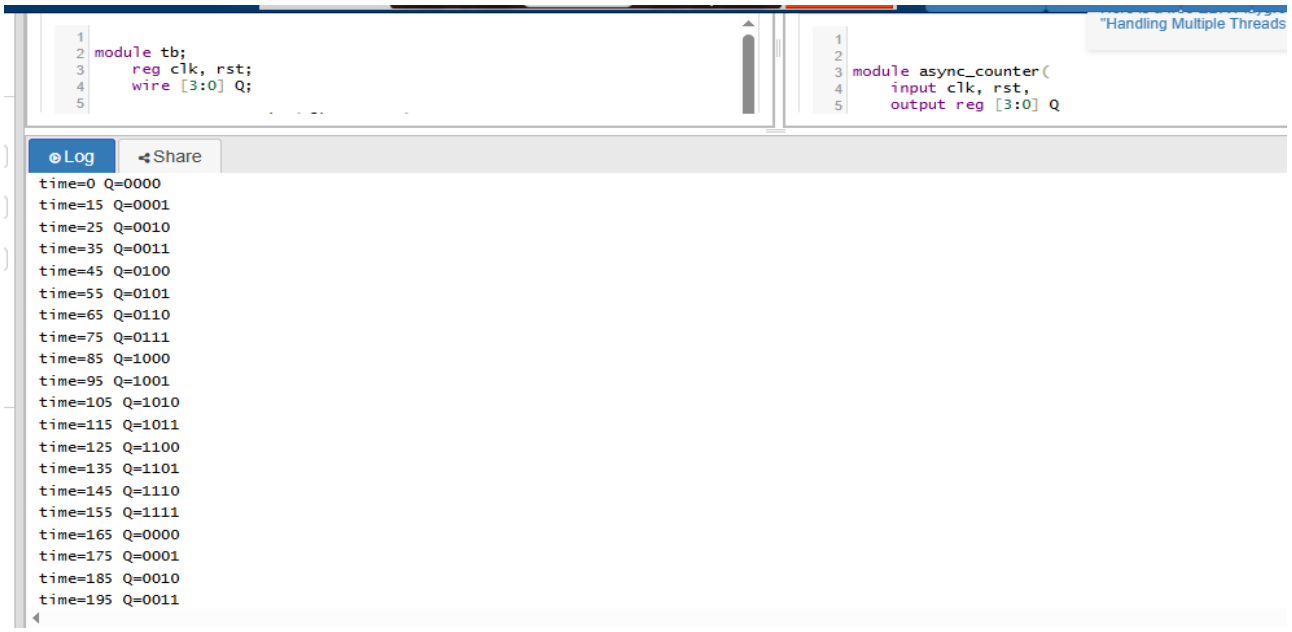
   async_counter dut(clk, rst, Q);

   always #5 clk = ~clk;

   initial begin
      clk = 0;
      rst = 1;
      #10;
      rst = 0;
      #200;
      $finish;
   end

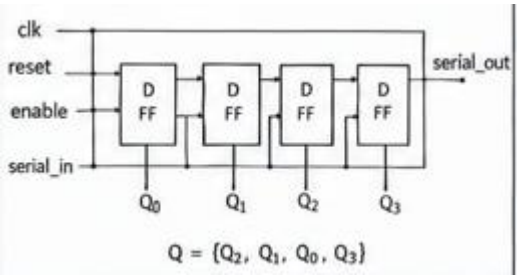
   initial
      $monitor("time=%0t Q=%b", $time, Q);
endmodule
```

SIMULATION RESULT:



10. Shift Register

DESIGN: A 4-bit shift register stores four bits and shifts the data by one position on every rising clock edge. The serial input enters the first flip-flop, and the last flip-flop provides the serial output.



Clock pulse	Serial input (SI)	Output Q
Reset	0	0000
1	1	0001
2	0	0010
3	1	0101
4	1	1011
5	0	0110

Verilog HDL design code:

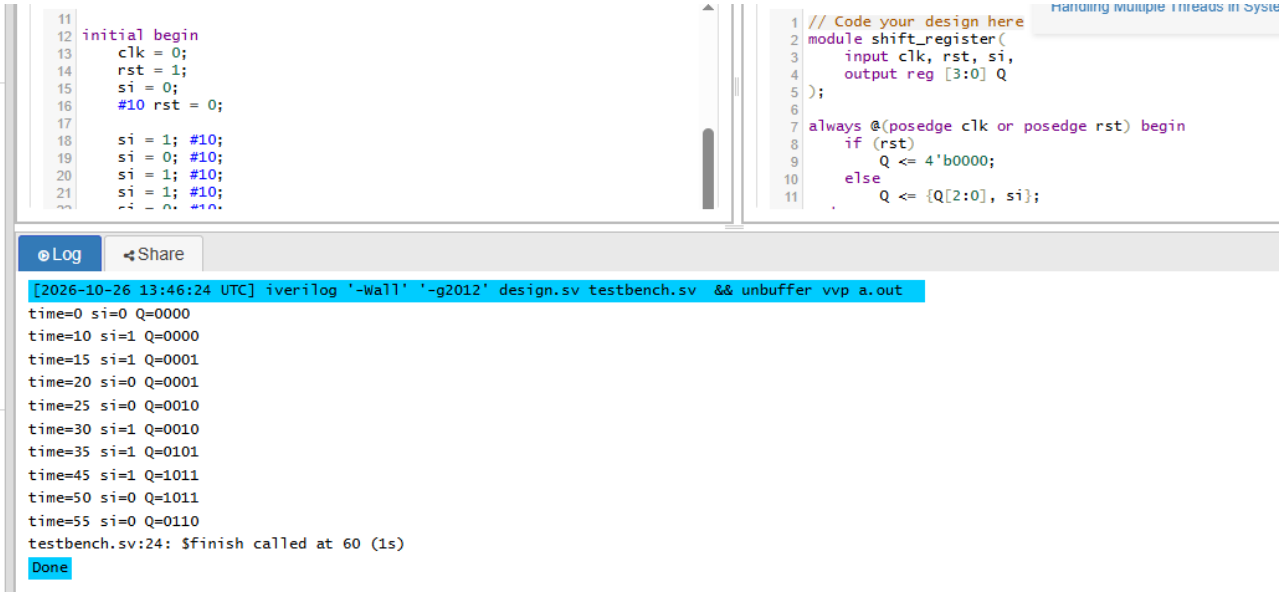
```
module shift_register(
   input clk, rst, si,
   output reg [3:0] Q
);
always @(posedge clk or posedge rst) begin
   if (rst)
      Q <= 4'b0000;
   else
      Q <= {Q[2:0], si};
end
endmodule
```

TESTBENCH CODE:

```
module tb;
reg clk, rst, si;
wire [3:0] Q;
shift_register dut(clk, rst, si, Q);
always #5 clk = ~clk;
```

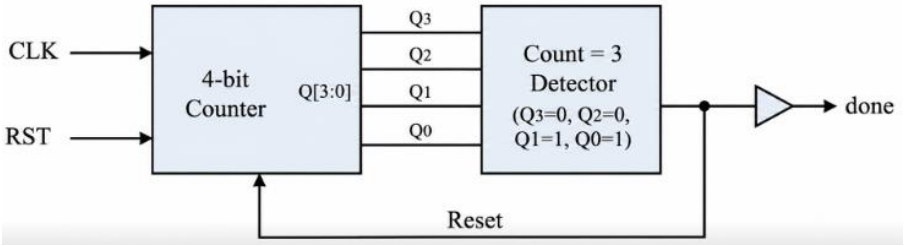
```
initial begin
    clk = 0;
    rst = 1;
    si = 0;
    #10 rst = 0;
    si = 1; #10;
    si = 0; #10;
    si = 1; #10;
    si = 1; #10;
    si = 0; #10;
    $finish;
end
initial
    $monitor("time=%0t si=%b Q=%b", $time, si, Q);
endmodule
```

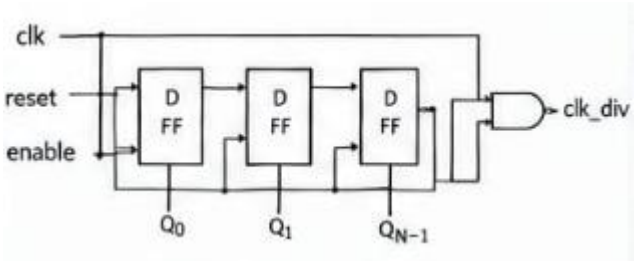
SIMULATION RESULT:



11. Divide by N-counter

DESIGN: A divide-by-N counter counts incoming clock pulses and generates an output event after every N pulses. In this example, the counter uses (N=4), so the done output becomes high for one clock cycle every four rising edges.





Clock pulse	Count before edge	Count after edge	done
Reset	—	0	0
1	0	1	0
2	1	2	0
3	2	3	0
4	3	0	1
5	0	1	0
6	1	2	0
7	2	3	0
8	3	0	1

Verilog HDL design code:

```
module divide_by_n #(
   parameter N = 4
)(
   input clk, rst,
   output reg done
);
reg [3:0] count;
always @(posedge clk or posedge rst) begin
   if (rst) begin
```

```
      count <= 0;
      done  <= 0;
   end
   else if (count == N-1) begin
      count <= 0;
      done  <= 1;
   end
   else begin
      count <= count + 1'b1;
      done  <= 0;
   end
end
endmodule
```

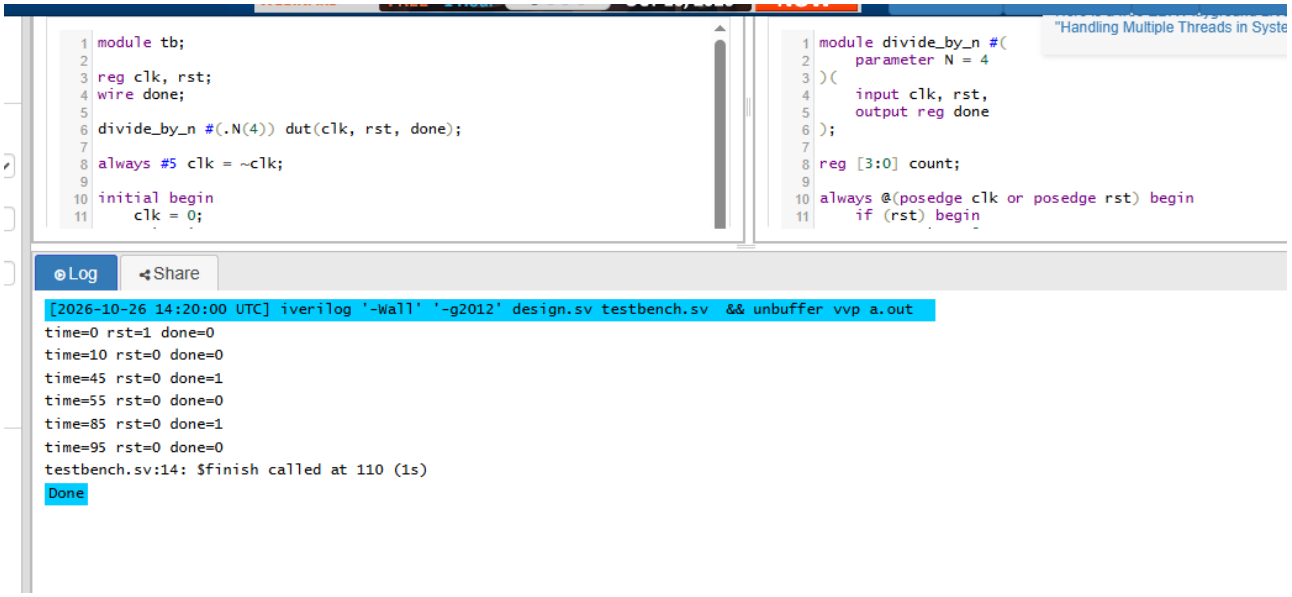
TESTBENCH CODE:

```
module tb;

reg clk, rst;
wire done;
divide_by_n #(.N(4)) dut(clk, rst, done);
always #5 clk = ~clk;
initial begin
   clk = 0;
   rst = 1;
   #10 rst = 0;
   #100 $finish;
end
initial
   $monitor("time=%0t rst=%b done=%b",
         $time, rst, done);

endmodule
```

SIMULATION RESULT:



```
module tb;

reg clk, rst;
wire done;

divide_by_n #(.N(4)) dut(clk, rst, done);

always #5 clk = ~clk;

initial begin
    clk = 0;
```

```
module divide_by_n #(
    parameter N = 4
)(
    input clk, rst,
    output reg done
);

reg [3:0] count;

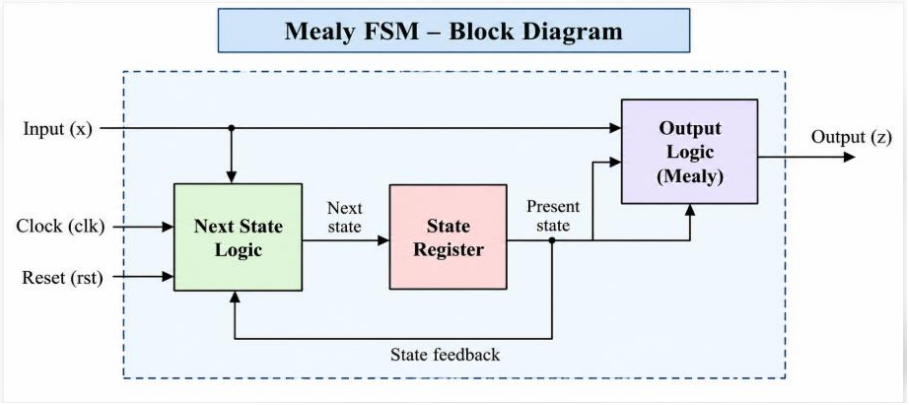
always @(posedge clk or posedge rst) begin
    if (rst) begin
```

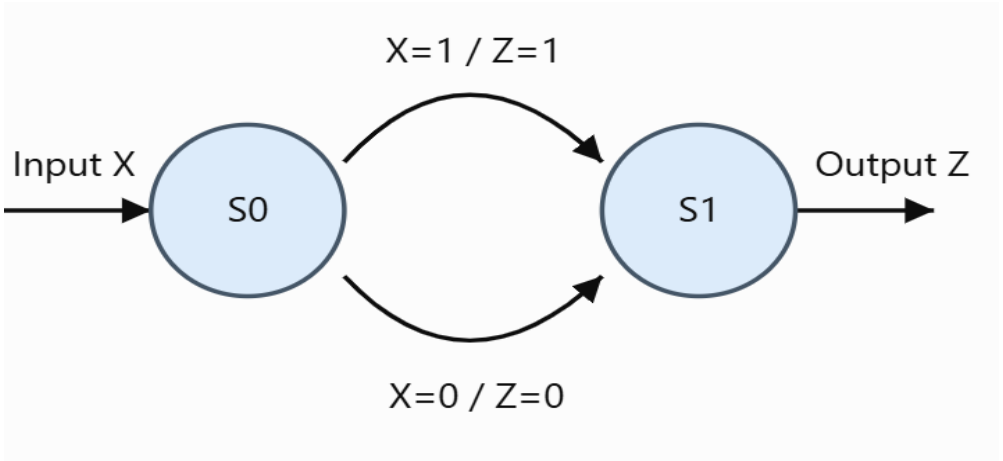
Log Share

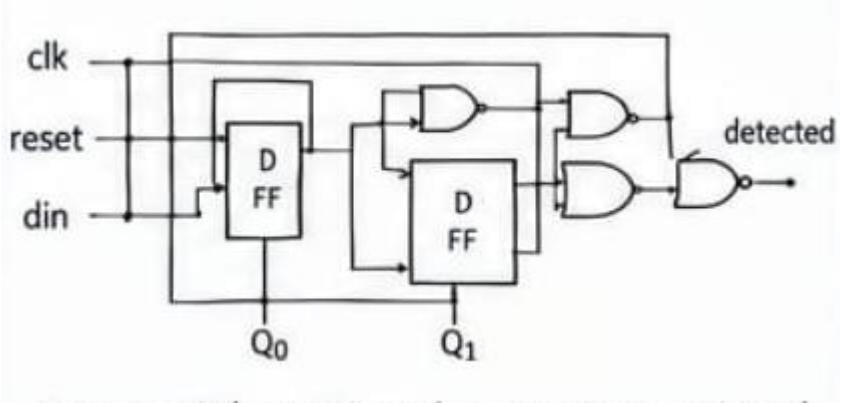
```
[2026-10-26 14:20:00 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
time=0 rst=1 done=0
time=10 rst=0 done=0
time=45 rst=0 done=1
time=55 rst=0 done=0
time=85 rst=0 done=1
time=95 rst=0 done=0
testbench.sv:14: $finish called at 110 (1s)
Done
```

12. Mealy FSM

DESIGN: A Mealy finite-state machine produces its output based on the present state and current input. Its output can change immediately when the input changes, without waiting for another clock edge.







Clock cycle	Input X	Output Z
Reset	0	0
1	1	1
2	0	0
3	1	1
4	1	1
5	0	0

Verilog HDL design code:

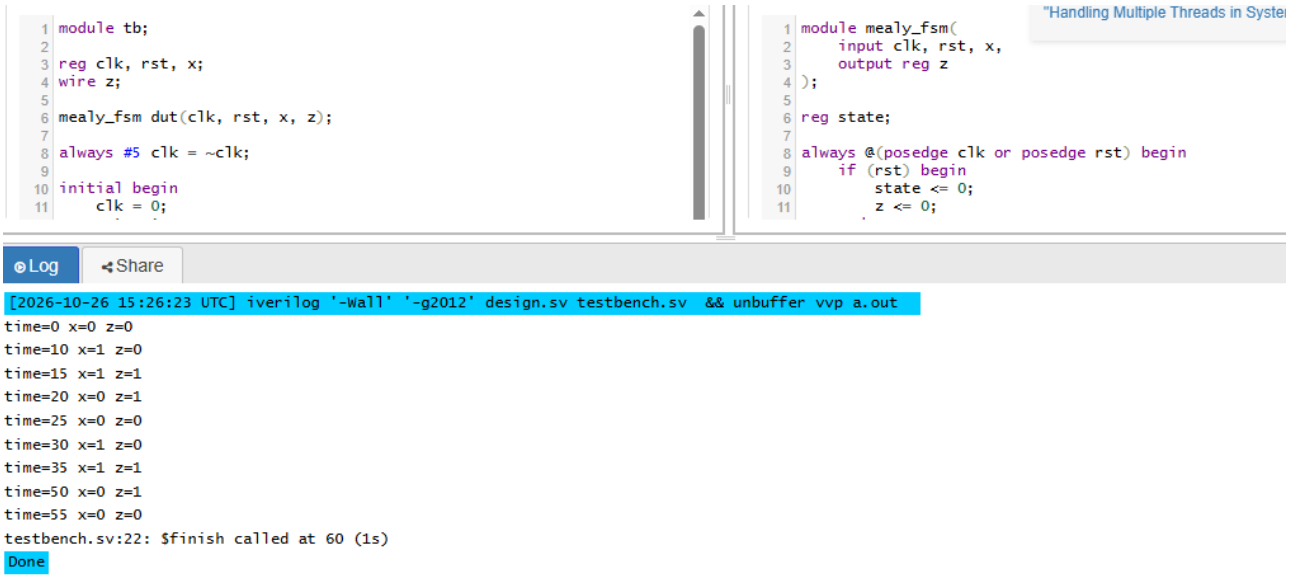
```
module mealy_fsm(
   input clk, rst, x,
   output reg z);
reg state;
always @(posedge clk or posedge rst) begin
   if (rst) begin
      state <= 0;
      z <= 0;
   end
   else begin
      case (state)
         0: begin
            if (x) begin
 state <= 1;
               z <= 1;
            end
            else begin
               state <= 0;
               z <= 0;
            end
         end
         1: begin
            if (x) begin
               state <= 1;
               z <= 1;
            end
            else begin
               state <= 0;
               z <= 0;
            end
```

```
        end
      endcase
   end
end
endmodule
```

TESTBENCH CODE:

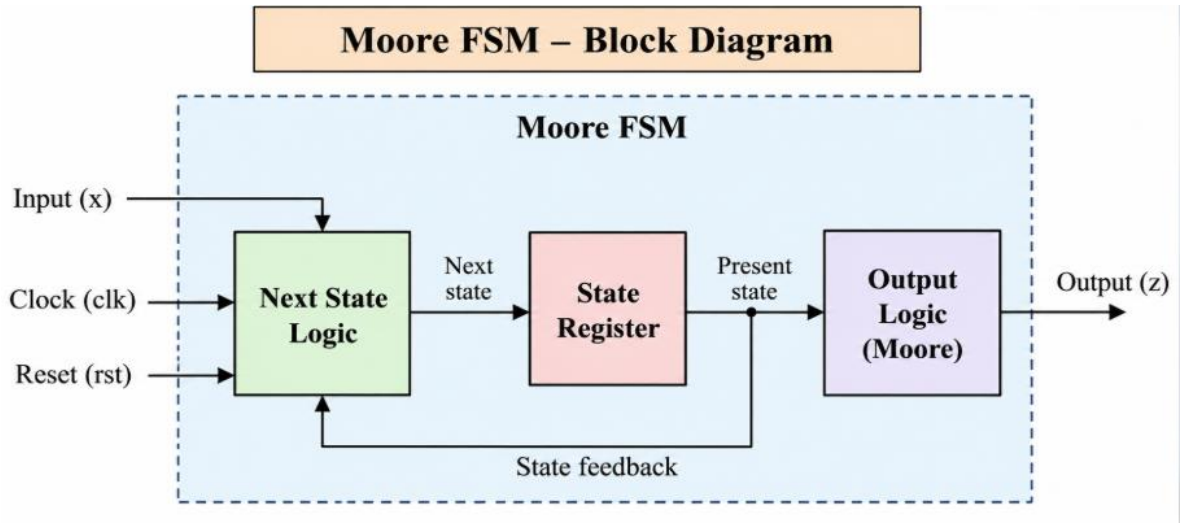
```
module tb;
reg clk, rst, x;
wire z;
mealy_fsm dut(clk, rst, x, z);
always #5 clk = ~clk;
initial begin
   clk = 0;
   rst = 1;
   x = 0;
   #10 rst = 0;
   x = 1; #10;
   x = 0; #10;
   x = 1; #10;
   x = 1; #10;
   x = 0; #10;
   $finish;
end
initial
   $monitor("time=%0t x=%b z=%b",
         $time, x, z);
endmodule
```

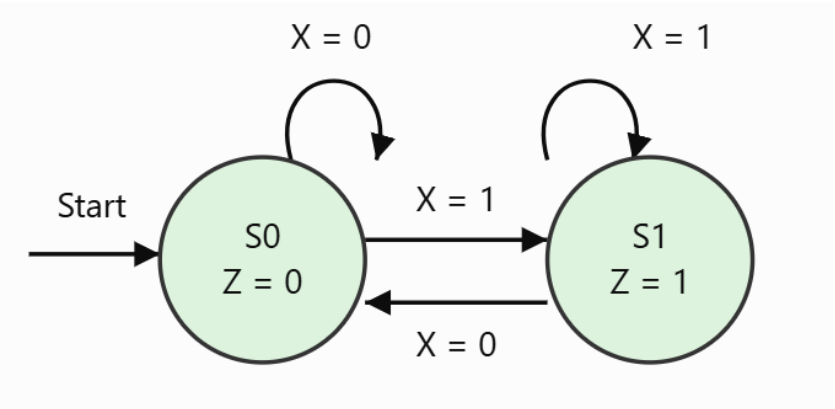
SIMULATION RESULT:



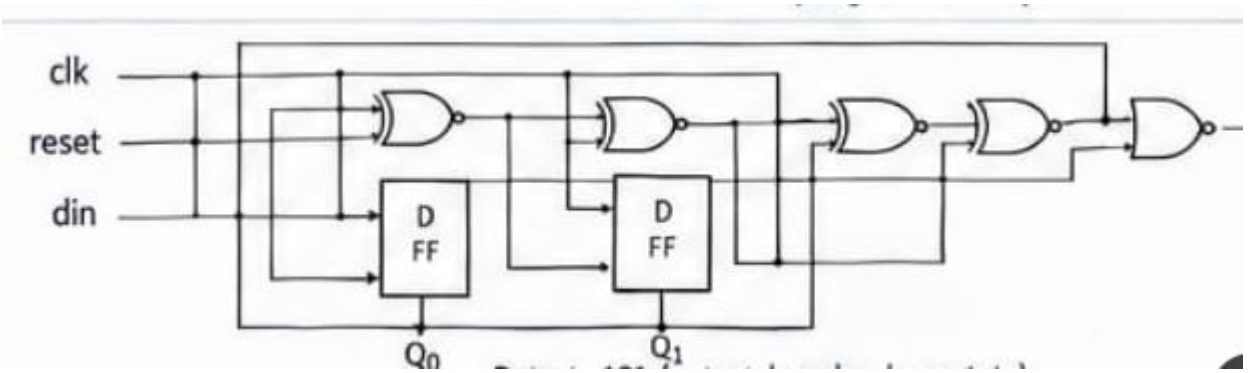
13. Moore FSM

DESIGN: A Moore FSM produces its output based only on the present state. The output changes when the state changes, usually at a clock edge.





Clock cycle	Input X	Present state	Output Z
Reset	0	S0	0
1	1	S0	0
2	0	S1	1
3	1	S0	0
4	1	S1	1
5	0	S1	1



Verilog HDL design code:

```
module moore_fsm(
   input clk, rst, x,
   output reg z
);

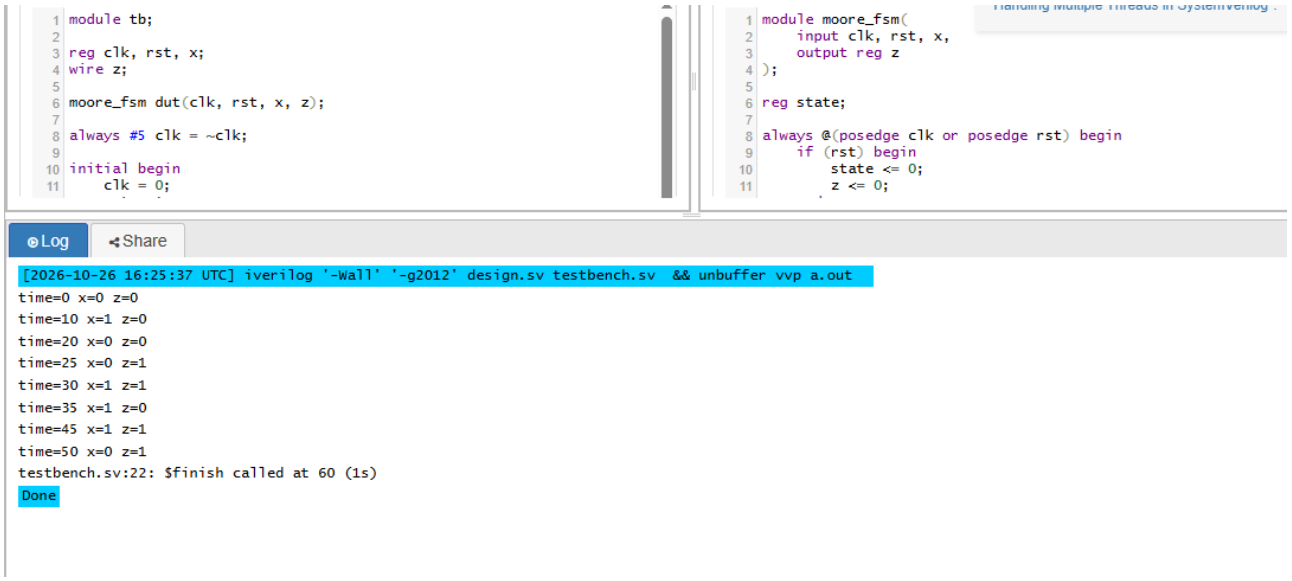
reg state;
always @(posedge clk or posedge rst) begin
   if (rst) begin
      state <= 0;
      z <= 0;
   end
   else begin
      case (state)
         0: begin
            z <= 0;
            if (x)
               state <= 1;
            else
               state <= 0;
         end
         1: begin
            z <= 1;
            if (x)
               state <= 1;
            else
               state <= 0;
         end
      endcase
   end
end
endmodule
```

TESTBENCH CODE:

```
module tb;
```

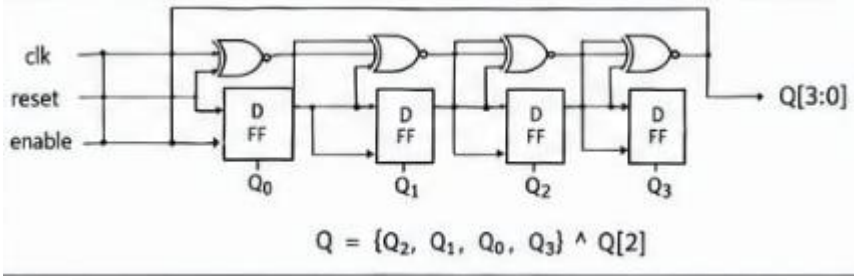
```
reg clk, rst, x;
wire z;
moore_fsm dut(clk, rst, x, z);
always #5 clk = ~clk;
initial begin
   clk = 0;
   rst = 1;
   x = 0;
   #10 rst = 0;
   x = 1; #10;
   x = 0; #10;
   x = 1; #10;
   x = 1; #10;
   x = 0; #10;
   $finish;
end
initial
   $monitor("time=%0t x=%b z=%b", $time, x, z);
endmodule
```

SIMULATION RESULT:



14. Random Sequence Generator (LFSR)

DESIGN: An LFSR generates a pseudo-random binary sequence by shifting the register contents and feeding back the XOR of selected bits. A non-zero initial state is required to generate a changing sequence.



Clock pulse	Output Q
Reset	0001
1	0011
2	0111
3	1111
4	1110
5	1101
6	1010
7	0101
8	1011

Verilog HDL design code:

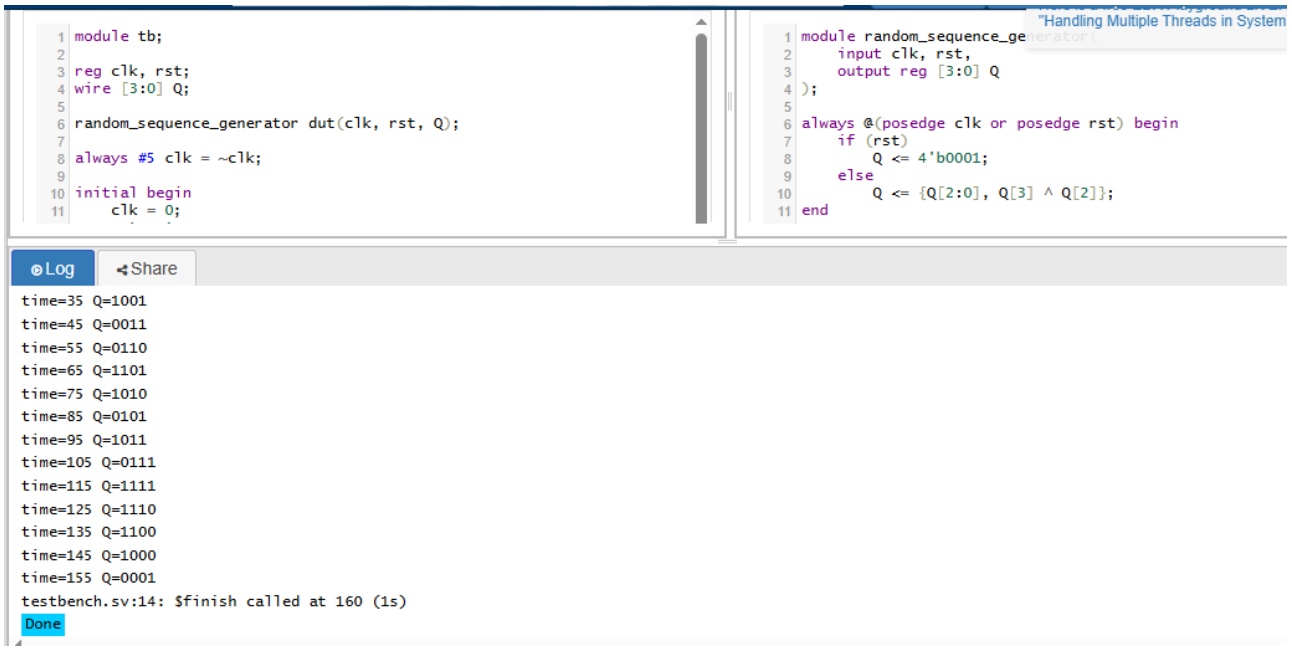
```
module random_sequence_generator(
   input clk, rst,
   output reg [3:0] Q
);
always @(posedge clk or posedge rst) begin
   if (rst)
      Q <= 4'b0001;
   else
      Q <= {Q[2:0], Q[3] ^ Q[2]};
end
endmodule
```

TESTBENCH CODE:

```
module tb;
```

```
reg clk, rst;
wire [3:0] Q;
random_sequence_generator dut(clk, rst, Q);
always #5 clk = ~clk;
initial begin
   clk = 0;
   rst = 1;
   #10 rst = 0;
   #150 $finish;
end
initial
   $monitor("time=%0t Q=%b", $time, Q);
endmodule
```

SIMULATION RESULT:



CONCLUSION

The designs were implemented and simulated using EDA Playground and their functionality was verified through simulation results.